Swahili Tutor Backend: Memory System and Session Pipeline

Introduction

This document describes the core components of the Swahili Tutor backend responsible for managing conversation state and session persistence. We focus on two key files: `memory.py` which implements intelligent conversation memory with cost controls, and `store.py` which manages session persistence. Together, these components form a pipeline that enables efficient, scalable tutoring sessions.

The Session Store (`store.py`)

The `store.py` module provides a development-grade in-memory session store designed for prototyping. Its primary responsibilities include:

Core Functionality

- Session Storage: Maintains a dictionary of `Session` objects keyed by session ID
- Thread Safety: Uses locks to ensure safe concurrent access
- Basic Operations:
  - `save(session)`: Persists or updates a session
  - `get(session_id)`: Retrieves a session by ID
  - `upsert_history(session_id, entries)`: Updates only the history portion of a session

Current Limitations

"Dev-grade in-memory session store. Lost on restart and single-process only. Swap for Postgres/Redis once you move past prototyping."

This implementation is suitable for development and testing but requires replacement with a persistent, distributed store (such as PostgreSQL or Redis) for production deployment.

The Memory System (`memory.py`)

The `memory.py` module implements a sophisticated conversation memory system built on LlamaIndex, designed to manage token usage while preserving important contextual information for language learning.

Key Components

1. ErrorCache

Tracks repeated error patterns to provide efficient feedback:

- Maintains a dictionary of `(error_type, root)` pattern occurrences
- Returns `True` when a pattern has been seen before (indicating a repeat error)
- Enables the system to provide brief reminders instead of full re-teachings for repeated mistakes
- Significant token saver for learners who commonly repeat specific errors

2. SummarizingMemory

Extends `ChatMemoryBuffer` with periodic summarization:

- Configuration: Token limit, summarization frequency, and recent turns to keep
- Mechanism:
  - Counts user turns
  - Every Nth user turn (configurable), summarizes older conversation history
  - Summary focuses on: recurring grammar mistakes and topics covered (2-3 sentences)
  - Keeps recent turns verbatim (configurable number)
  - Replaces old history with the summary note marked by `[Early-conversation summary]`

- Benefits:
  - Prevents long sessions from paying full token cost for early turns
  - Buffer's token limit acts as a safety net for single-turn protection
  - Preserves pedagogically relevant information through targeted summarization

3. MemoryRegistry

Manages per-session memory and error cache instances:
- Thread-safe caching with locks
- Rebuilds memory from persisted `session.history` snapshot
- Provides cleanup functionality for sessions
- Owns one memory + error cache per session

Workflow Integration

When a session begins:
1. The registry loads history from the store
2. Rebuilds the `SummarizingMemory` instance from the history snapshot
3. As conversation progresses:
   - User turns increment the turn counter
   - At configured intervals, older history is summarized
   - Error patterns are tracked and checked for repetition
   - Memory buffer provides context for LLM interactions

The Pipeline: Workflows, Memory, and Store

The tutoring workflow follows these steps:

1. Session Initialization
   - Workflow retrieves session from `store.py` using session ID
   - If session exists, history is extracted; otherwise, new session created
   - `MemoryRegistry` rebuilds `SummarizingMemory` from history snapshot
2. Turn Processing
   - User input received and wrapped as `ChatMessage`
   - Message added to memory buffer via `SummarizingMemory.put()`
   - User turns increment internal counter
3. Periodic Summarization
   - When turn counter reaches multiple of `summarize_every`:
     - Older history (beyond `keep_recent` threshold) extracted
     - LLM summarizes this history focusing on grammar mistakes and topics
     - Memory buffer reset with: summary note + recent verbatim turns
     - Summary stored as special system message
4. Error Tracking
   - For each detected error, `ErrorCache.check_and_record()` called
   - If repeat error detected (`True` returned), workflow can provide brief reminder
   - Otherwise, full re-teaching proceeds
5. Persistence
   - After processing, updated session history saved via `store.upsert_history()`
   - History stored as structured list of dictionaries (role/content pairs)
   - Enables session recovery across restarts (when persistent store implemented)

Technical Characteristics

Memory Efficiency

- Token Cost Reduction: Summarization prevents linear token growth with session length
- Targeted Retention: Preserves pedagogically relevant information (mistakes, topics) while compressing conversational filler
- Error Feedback Optimization: Reduces redundant explanations for common mistakes

Scalability Considerations

- Current Limitation: In-memory store prevents horizontal scaling
- Migration Path: Designed for straightforward replacement with Redis/PostgreSQL
- Memory Bounds: Token limits prevent unbounded memory growth per session
- Thread Safety: Lock-based synchronization supports concurrent session handling

Pedagogical Design

- Context Preservation: Summary focuses on learning-relevant content (mistakes, topics)
- Adaptive Feedback: Error tracking enables personalized remediation
- Conversation Flow: Recent verbatim turns maintain immediate conversational context

Future Improvements

For `store.py`

1. Persistent Storage Layer
   - Implement PostgreSQL backend with connection pooling
   - Add Redis caching layer for hot sessions
   - Implement session TTL and automatic cleanup
2. Enhanced Session Management
   - Add session locking for distributed environments
   - Implement session versioning for conflict detection
   - Add backup/restore capabilities for session data
3. Observability
   - Add metrics collection (session count, memory usage, hit rates)
   - Implement health check endpoints
   - Add structured logging for session operations

For `memory.py`

1. Advanced Summarization Strategies
   - Implement importance-based summarization (retain key learning moments)
   - Add extractive summarization for factual retention alongside abstractive
   - Dynamic summary length based on session difficulty/progress
2. Sophisticated Error Tracking
   - Error categorization by Swahili grammar topics (noun classes, verb tenses)
   - Spaced repetition scheduling for error remediation
   - Error severity weighting and prioritization
   - Tracking of error resolution success rates
3. Memory Analytics
   - Token savings measurement from summarization/error caching
   - Learning progress inference from memory patterns
   - A/B testing framework for memory strategies

- Export capabilities for memory analysis and research

4. Swahili-Specific Enhancements
   - Integration with Swahili morphological analyzers for better error detection
   - Vocabulary tracking and spaced review scheduling
   - Cultural context retention and recall
   - Pronunciation pattern tracking (if audio components added)

Cross-Cutting Improvements

1. Testing and Reliability
   - Comprehensive unit tests for edge cases
   - Fallback mechanisms for LLM summarization failures
   - Circuit breaker patterns for external dependencies
   - Chaos testing for memory/store interactions
2. Performance Optimization
   - Asynchronous summarization to avoid blocking user turns
   - Memory compression techniques for stored histories
   - Batch processing opportunities for multiple sessions
   - Profiling and optimization of hot code paths
3. Integration Enhancements
   - Automatic persistence hooks between memory and store
   - Memory format versioning for backward compatibility
   - Shared memory capabilities for related sessions (same learner)
   - Import/export functionality for session migration and backup

Conclusion

The `memory.py` and `store.py` modules form a critical foundation for the Swahili Tutor backend's ability to conduct efficient, personalized tutoring sessions. The memory system intelligently balances contextual preservation with token efficiency through targeted summarization and error tracking, while the store provides the session persistence layer necessary for scalable deployment.

While the current implementation serves well for development and prototyping, the documented limitations and suggested improvements outline a clear path toward a production-ready system capable of handling thousands of concurrent learners while maintaining the pedagogical effectiveness that makes this tutor valuable for Swahili language education.

By implementing the proposed enhancements—particularly migrating to a persistent, distributed store and advancing the memory system with Swahili-specific pedagogical intelligence—the backend will be well-positioned to support a growing user base while delivering increasingly effective personalized instruction.